МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №2

з курсу: „ Інтернет речей та вбудовані системи”

виконав студент 5го курсу
групи КН-51мп
Сидько Н.С.

КИЇВ
2025

Мета роботи: програмування таймерів та контролера прямого доступу до пам’яті у САПР Proteus

Хід роботи

1. Налаштування незалежного сторожового таймера (IWDG)

Для реалізації цього завдання було створено програму, яка налаштовує незалежний сторожовий таймер (IWDG) на мікроконтролері STM32F103C6.

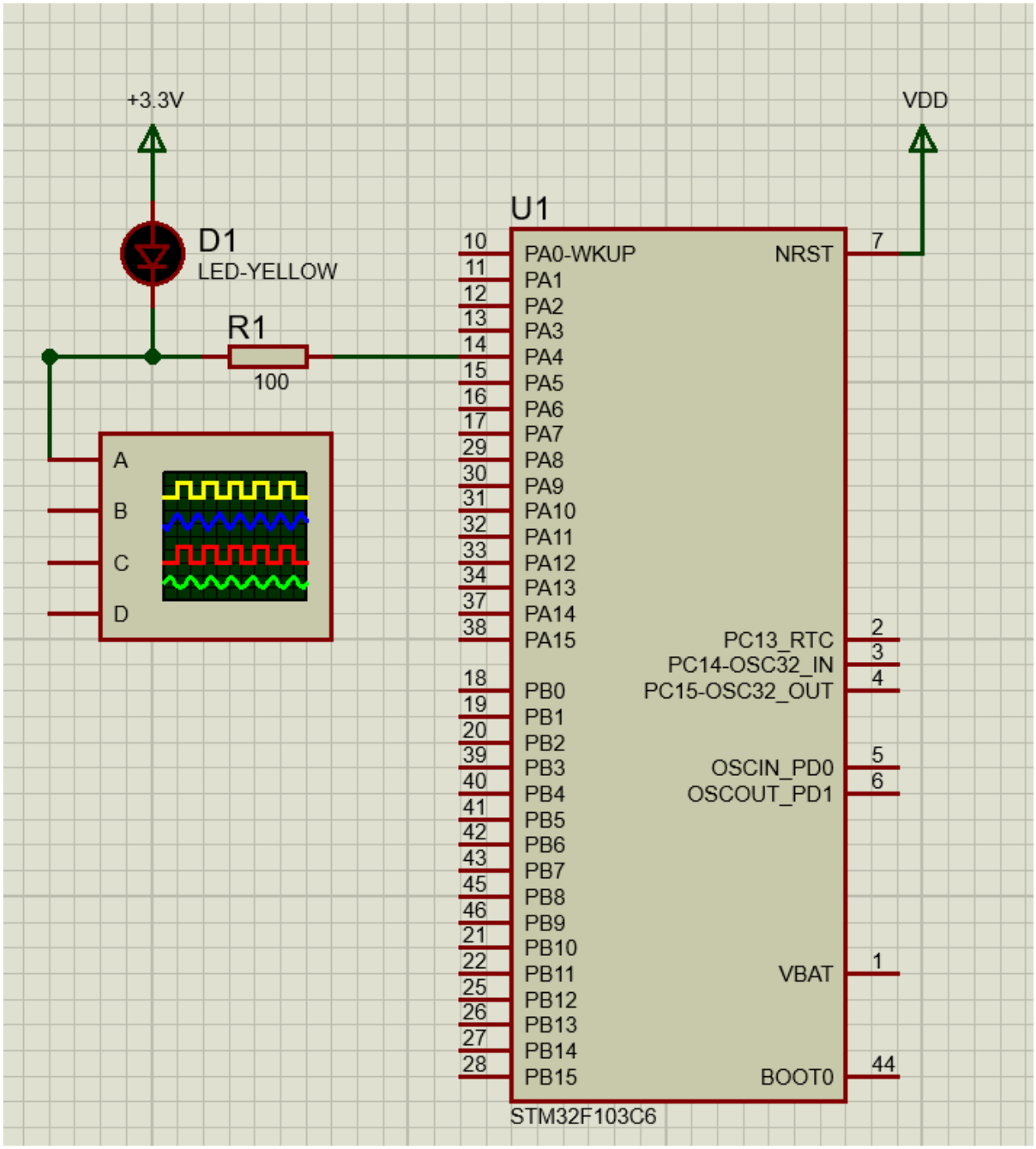


Рис.1 – Схема налаштування незалежного сторожового таймера

На першому етапі було активовано тактування порту GPIOA і налаштовано пін PA4 як цифровий вихід. Через цей пін підключено світлодіод, який короткочасно загоряється при кожному старті програми, що дозволяє наочно визначити момент перезавантаження мікроконтролера.

Конфігурація IWDG здійснювалась у кілька кроків. Спочатку записом ключа 0x5555 до регістру IWDG->KR знімався захист від зміни налаштувань. Для визначення часу скидання обрано дільник частоти (prescaler) рівний 32, що відповідає запису значення 3 у регістр IWDG->PR. Значення перезавантаження лічильника (Reload) було обчислено таким чином, щоб забезпечити таймаут близько 105 мс. При частоті внутрішнього LSI-осцилятора приблизно 40 кГц, необхідне значення становило (0.105 с * 40000 Гц) / 32 ≈ 131. Це число записано у регістр IWDG->RLR. Після цього лічильник сторожового таймера було перезавантажено ключем 0xAAAA і активовано командою 0xCCCC.

У головному циклі while(1) програма не здійснює скидання таймера вручну. Внаслідок цього, по завершенню таймауту 105 мс IWDG автоматично ініціює апаратне перезавантаження мікроконтролера. Кожен цикл запуску програми супроводжується коротким спалахом світлодіода на PA4, що дозволяє наочно контролювати інтервали перезавантаження.

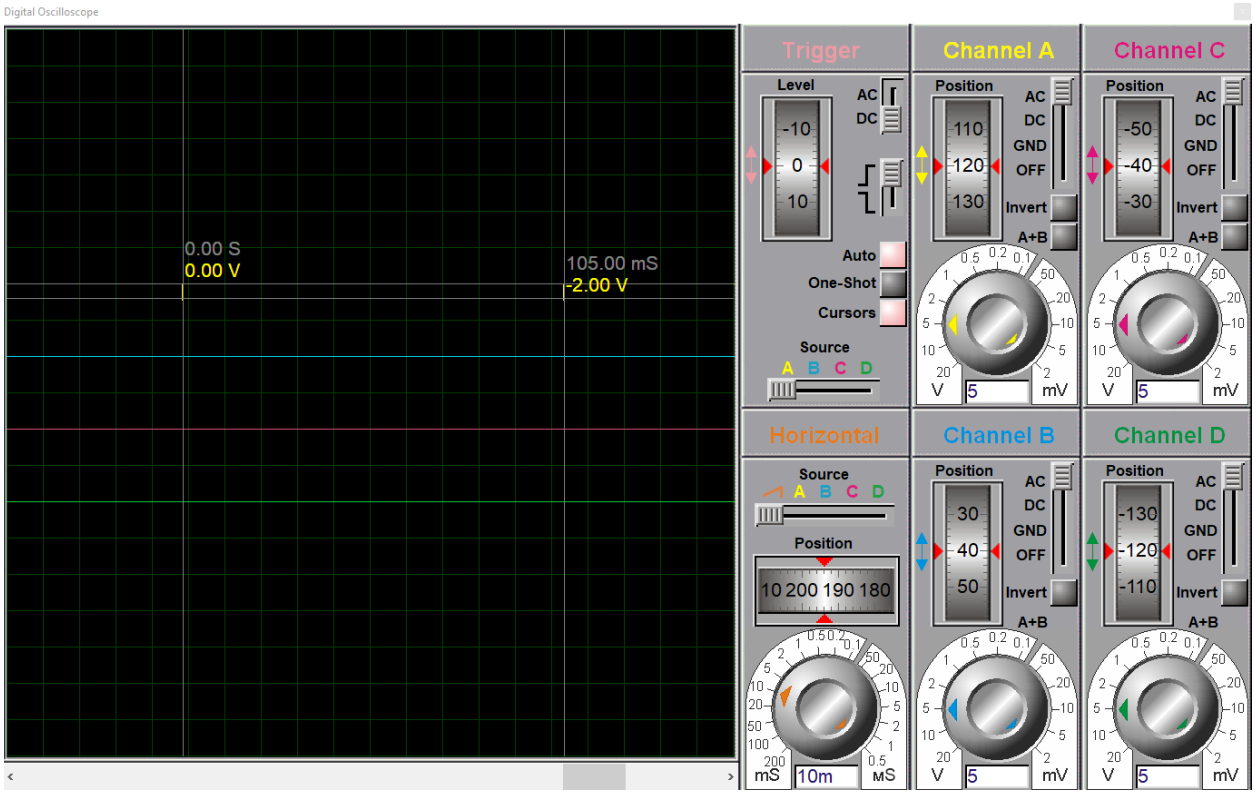


Рис. 2 – Відображення таймінгів скидання на осцилографі

```
#include "stm32f1xx.h"

int main(void) {

    // 1. Увімкнення тактування для порту GPIOA
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;
```

```
    // 2. Налаштування піна PA4 як цифрового виходу (push-pull, 10MHz)
    // Спочатку очищуємо поточні налаштування для MODE та CNF
    GPIOA->CRL &= ~(GPIO_CRL_CNF4 | GPIO_CRL_MODE4);
     // Встановлюємо режим: MODE4 = 01 (Output mode, max speed 10 MHz), CNF4 = 00 (Push-pull)
    GPIOA->CRL |= GPIO_CRL_MODE4_0;

    // 3. Короткочасне увімкнення світлодіода при старті програми
    // Це дозволяє візуально визначити момент запуску МК
    GPIOA->BSRR = GPIO_BSRR_BS4;  // Увімкнути світлодіод

    // 4. Налаштування незалежного сторожового таймера IWDG
    // Потрібний час скидання: 5 * N = 5 * 21 = 105 мс
    // Тактова частота LSI ~ 40 кГц
    // Формула таймауту: T = (Prescaler * Reload_Value) / F_LSI

    // Крок 1: Зняти захист регістрів IWDG_PR та IWDG_RLR
    IWDG->KR = 0x5555;

    // Крок 2: Встановити дільник частоти (prescaler)
    // Обираємо Prescaler = 32 (значення в регістрі PR = 3)
    IWDG->PR = 3; // Дільник = 32

   // Крок 3: Встановити значення перезавантаження лічильника (Reload_Value)
    // Розрахунок: Reload_Value = (T * F_LSI) / Prescaler
    // Reload_Value = (0.105 с * 40000 Гц) / 32 ? 131
    IWDG->RLR = 131;

      // Крок 4: Перезавантажити лічильник IWDG, щоб застосувалися нові значення
    IWDG->KR = 0xAAAA;

    // Крок 5: Запустити таймер IWDG
    IWDG->KR = 0xCCCC;

    // 5. Нескінченний цикл без скидання watchdog'a
     // Програма залишатиметься тут доти, доки IWDG не викличе апаратне скидання МК
    while (1) {
       // Очікуємо на скидання мікроконтролера
    }
}
```

2. Створення обробника переривання системного таймера SysTick.

Варіант	$T_{ввімк}$, мс	$T_{вимк}$, мс
21	50	15

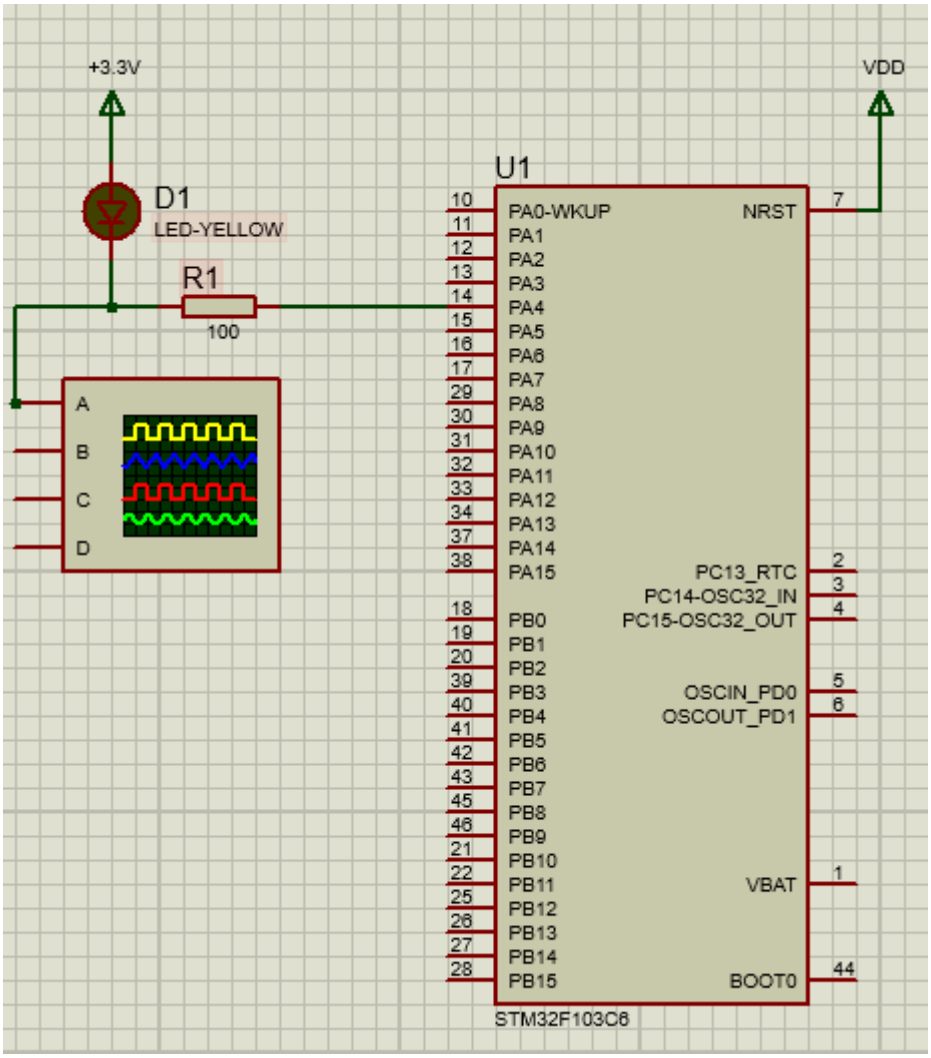


Рис. 3 – Схема для реалізації мигання світлодіодом

Для реалізації керування світлодіодом у неблокуючому режимі було використано системний таймер SysTick, який дозволяє організувати часові інтервали без застосування функцій затримки. Таймер було сконфігуровано таким чином, щоб він формував переривання з інтервалом 1 мс. Це забезпечується завантаженням у регістр перезавантаження значення, що відповідає кількості тактів процесора за одну мілісекунду, а саме F_CPU / 1000.

Уся логіка перемикання стану світлодіода реалізована безпосередньо в обробнику переривання SysTick_Handler. Для цього було використано глобальну змінну з атрибутом volatile, яка виконує роль лічильника часу в мілісекундах та збільшується при кожному виклику обробника.

На основі значення цього лічильника визначається поточний режим роботи світлодіода, підключеного до виводу PA4:

- протягом перших 50 мс світлодіод перебуває в активному стані;
- у наступні 15 мс відбувається його вимкнення;

- після досягнення повної тривалості циклу (65 мс) лічильник обнуляється, що ініціює початок нового циклу керування.

Таким чином, перемикання світлодіода відбувається у фоновому режимі з використанням апаратних переривань, що не блокує виконання основної програми.

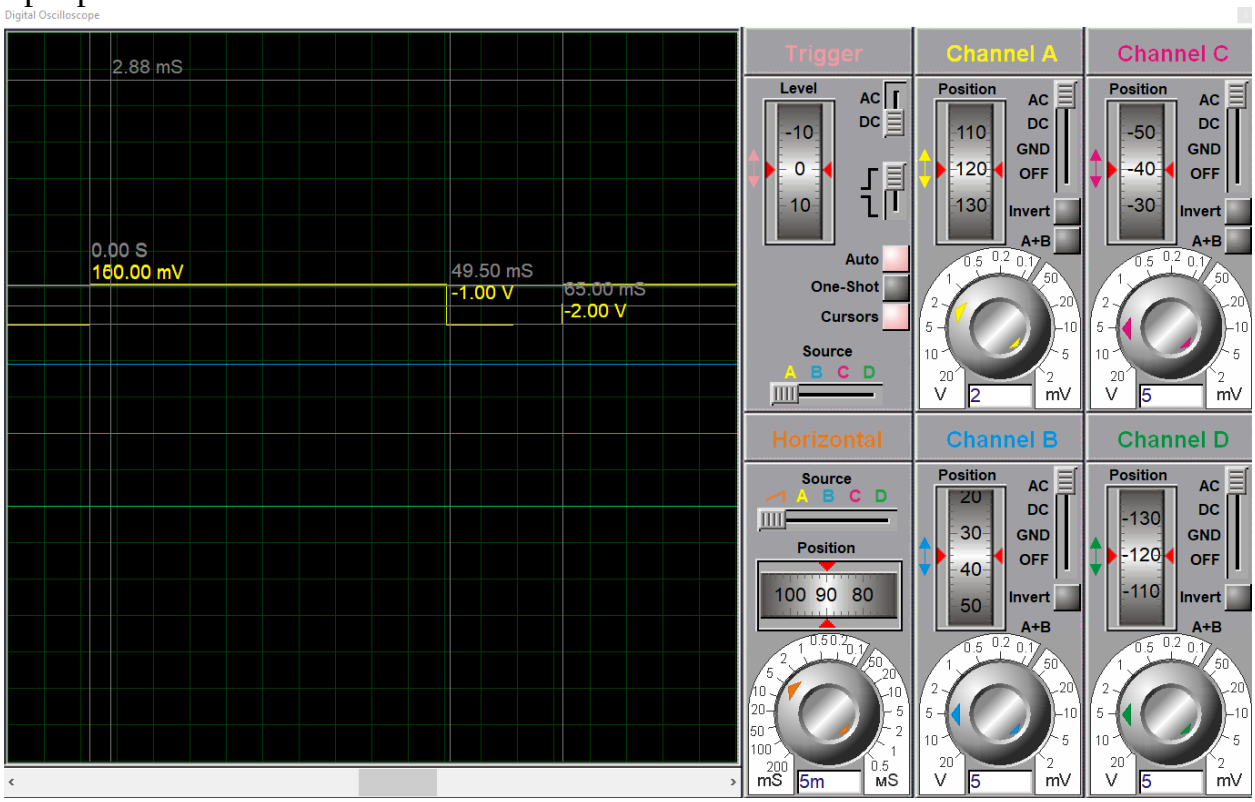


Рис. 4 – Відображення таймінгів мигання світлодіодом за допомогою SysTick

```
#include <stm32f1xx.h>

#define F_CPU 8000000        // Тактова частота МК - 8 МГц
#define LED_ON_TIME_MS  50    // Час увімкненого стану (50 мс)
#define LED_OFF_TIME_MS 15    // Час вимкненого стану (15 мс)
#define TOTAL_PERIOD_MS (LED_ON_TIME_MS + LED_OFF_TIME_MS) // Загальний період (65 мс)

// Глобальний лічильник мілісекунд
volatile unsigned int ms_counter = 0;

int main(void) {
    // 1. Увімкнення тактування порту GPIOA
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

    // 2. Налаштування піна PA4 як вихід push-pull, швидкість 10 МГц
     GPIOA->CRL &= ~(GPIO_CRL_CNF4 | GPIO_CRL_MODE4); // Очистити налаштування
```

```
   GPIOA->CRL |= GPIO_CRL_MODE4_0;              // Встановити Output mode 10 MHz, Push-pull

     // 3. Ініціалізація системного таймера SysTick для генерації переривань щомілісекунду
   SysTick_Config(F_CPU / 1000);

   // 4. Нескінченний цикл
   while (1) {
        // Основний цикл залишаємо порожнім, керування LED відбувається у SysTick
   }

   return 0;
}

// Обробник переривання SysTick (викликається кожну мілісекунду)
void SysTickHandler(void) {
   // Збільшення лічильника мілісекунд
   ms_counter++;

   // Керування світлодіодом:
   // 0 .. (LED_ON_TIME_MS-1) - LED увімкнено
   if (ms_counter < LED_ON_TIME_MS) {
      GPIOA->BSRR = GPIO_BSRR_BS4;  // Увімкнути LED (Set)
   }
   // LED_OFF_TIME_MS наступні мілісекунди - LED вимкнено
   else {
      GPIOA->BSRR = GPIO_BSRR_BR4;  // Вимкнути LED (Reset)
   }

   // Перевірка кінця періоду
   if (ms_counter >= TOTAL_PERIOD_MS) {
      ms_counter = 0; // Скидаємо лічильник для нового циклу
   }
}
```

3. Виклик переривання через певні інтервали часу
Інтервали часу наведені у табл. 2.

Варіант	$T_{ввімк1}$, мс	$T_{ввімк2}$, мс	$T_{ввімк3}$, мс	$T_{ввімк4}$, мс	$T_{ввімк5}$, мс
21	1.5	90	90	$30 \cdot 10^4$	2.2

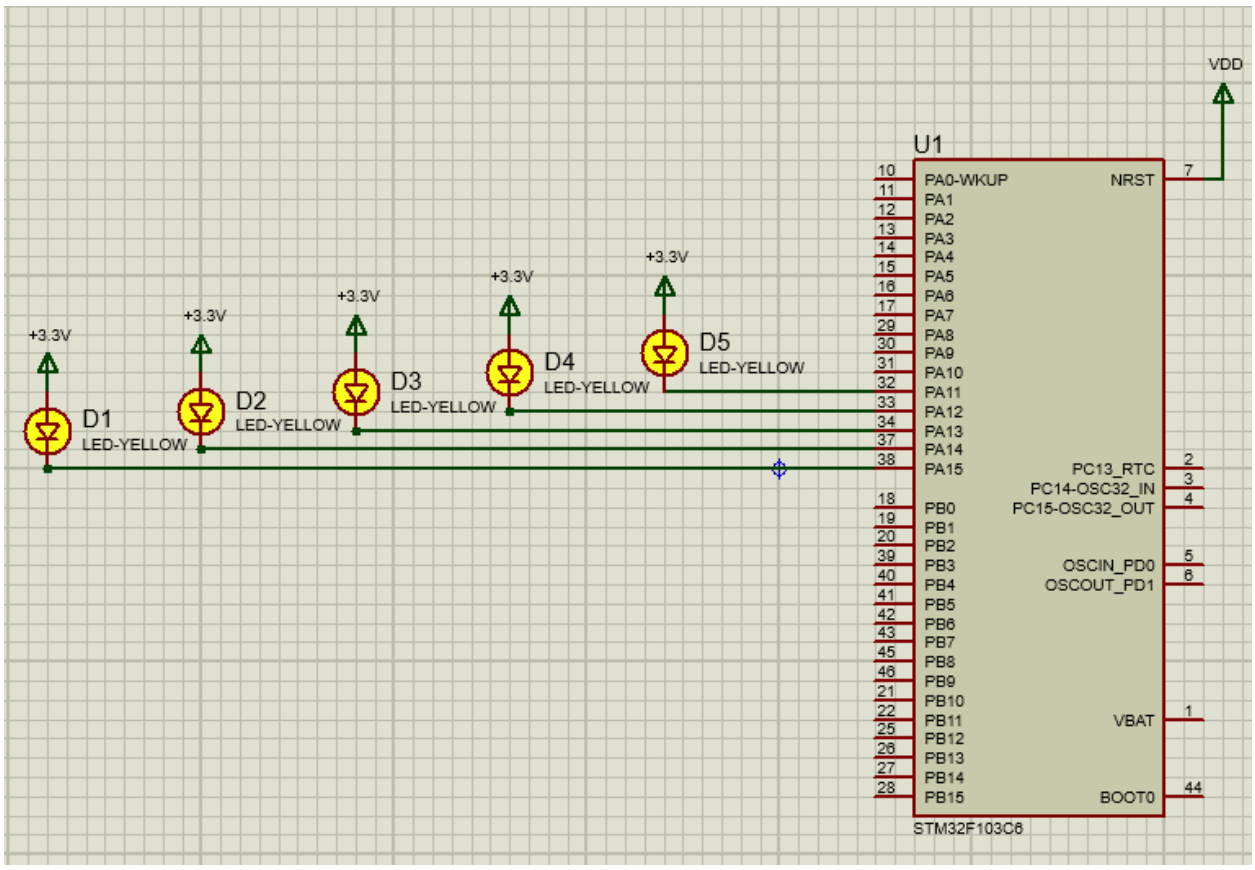


Рис. 5 – Схема для реалізації переривань з різними інтервалами

Для цього завдання було використано таймер загального призначення TIM2.

Його робота налаштована так, щоб один тик таймера відповідав 0.1 мс. Для керування часом між спрацьовуваннями переривання створено масив intervals, у якому зберігаються необхідні інтервали у кількості тиків таймера.

В обробнику переривань TIM2_IRQHandler реалізується послідовне включення світлодіодів на пинах PA11–PA15. Після кожного спрацьовування спочатку вимикаються всі світлодіоди, а потім активується поточний, що відповідає індексу interval_idx. Далі програмно відбувається перехід до наступного інтервалу з масиву, а при досягненні кінця списку індекс повертається на початок. Скидання прапора переривання в кінці обробника забезпечує коректну роботу таймера і дозволяє генерувати послідовність імпульсів із змінною тривалістю, керовану програмно.

```
#include "stm32f1xx.h"

#define F_APB1 24000000UL  // Частота шини APB1

// Інтервали в "тіках" по 0.1 мс
const uint32_t intervals[] = {15, 900, 900, 300, 22}; // Приклади: 1.5 мс, 90 мс, ...

volatile uint8_t interval_idx = 0;   // Індекс поточного інтервалу
volatile uint32_t tick_counter = 0;  // Лічильник тіків
```

```
// Кількість світлодіодів
#define LED_COUNT 5
#define LED_START_PIN 11  // Світлодіоди підключені до PA11–PA15

// Обробник переривання таймера TIM2
void TIM2_IRQHandler(void) {
   if (TIM2->SR & TIM_SR_UIF) {          // Перевірка прапору переповнення
      TIM2->SR &= ~TIM_SR_UIF;          // Скидання прапору переривання

      tick_counter++;                   // Інкремент лічильника тіків

      if (tick_counter >= intervals[interval_idx]) {
         // Вимикаємо всі світлодіоди
                  GPIOA->BSRR  =  (GPIO_BSRR_BR11  |  GPIO_BSRR_BR12  |
GPIO_BSRR_BR13 |
                   GPIO_BSRR_BR14 | GPIO_BSRR_BR15);

         // Включаємо поточний світлодіод
         if (interval_idx < LED_COUNT) {
            GPIOA->BSRR = (1 << (LED_START_PIN + interval_idx));
         }

         // Скидаємо лічильник
         tick_counter = 0;

         // Переходимо до наступного інтервалу
         interval_idx++;
         if (interval_idx >= LED_COUNT) interval_idx = 0;
      }
   }
}

// Ініціалізація таймера TIM2
void timer_init(void) {
    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;    // Увімкнення тактування
TIM2

   // Налаштування таймера: 0.1 мс тик
   TIM2->PSC = 23;   // F_APB1 / (PSC+1) = 1 МГц > 1 тик = 1 мкс
   TIM2->ARR = 99;   // 1 мкс * 100 = 0.1 мс

   TIM2->DIER |= TIM_DIER_UIE;   // Дозвіл переривання по переповненню
   TIM2->CNT = 0;                // Скидання лічильника
```

```
    NVIC_EnableIRQ(TIM2_IRQn);    // Дозвіл переривань у NVIC

    TIM2->CR1 |= TIM_CR1_CEN;     // Запуск таймера
}

// Ініціалізація GPIO для світлодіодів
void GPIO_Init(void) {
    // Увімкнення тактування GPIOA та AFIO
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_AFIOEN;

    // Вимикаємо JTAG, залишаємо SWD
    AFIO->MAPR |= AFIO_MAPR_SWJ_CFG_JTAGDISABLE;

    // PA11–PA15 як виходи push-pull 10 МГц
    GPIOA->CRH &= ~(GPIO_CRH_MODE11 | GPIO_CRH_CNF11 |
                    GPIO_CRH_MODE12 | GPIO_CRH_CNF12 |
                    GPIO_CRH_MODE13 | GPIO_CRH_CNF13 |
                    GPIO_CRH_MODE14 | GPIO_CRH_CNF14 |
                    GPIO_CRH_MODE15 | GPIO_CRH_CNF15);

    GPIOA->CRH |= (GPIO_CRH_MODE11_0 | GPIO_CRH_MODE12_0 |
                   GPIO_CRH_MODE13_0 | GPIO_CRH_MODE14_0 |
                   GPIO_CRH_MODE15_0);

    // Вимикаємо всі світлодіоди
                GPIOA->BSRR = (GPIO_BSRR_BR11 | GPIO_BSRR_BR12 |
GPIO_BSRR_BR13 |
                    GPIO_BSRR_BR14 | GPIO_BSRR_BR15);
}

int main(void) {
    GPIO_Init();     // Ініціалізація GPIO
    timer_init();    // Ініціалізація таймера

    while(1) {
        // Основний цикл порожній, управління світлодіодами через переривання
    }
}
```

4. Написати програму для вимірювання періода ШІМ сигналу. Для цього один з виходів таймера налаштовується на режим ШІМ, інший – на режим захоплення імпульсів ШІМ. Вхід і вихід таймера з’єднуються між собою. Таблиця входів виходів згідно з варіантом наведена у табл. 3.

Варіант	Вхід	Вихід	Період ШІМ, мкс	γ	ШІМ
21	TIM3, Ch1	TIM2, Ch1	30	30	Двостороння

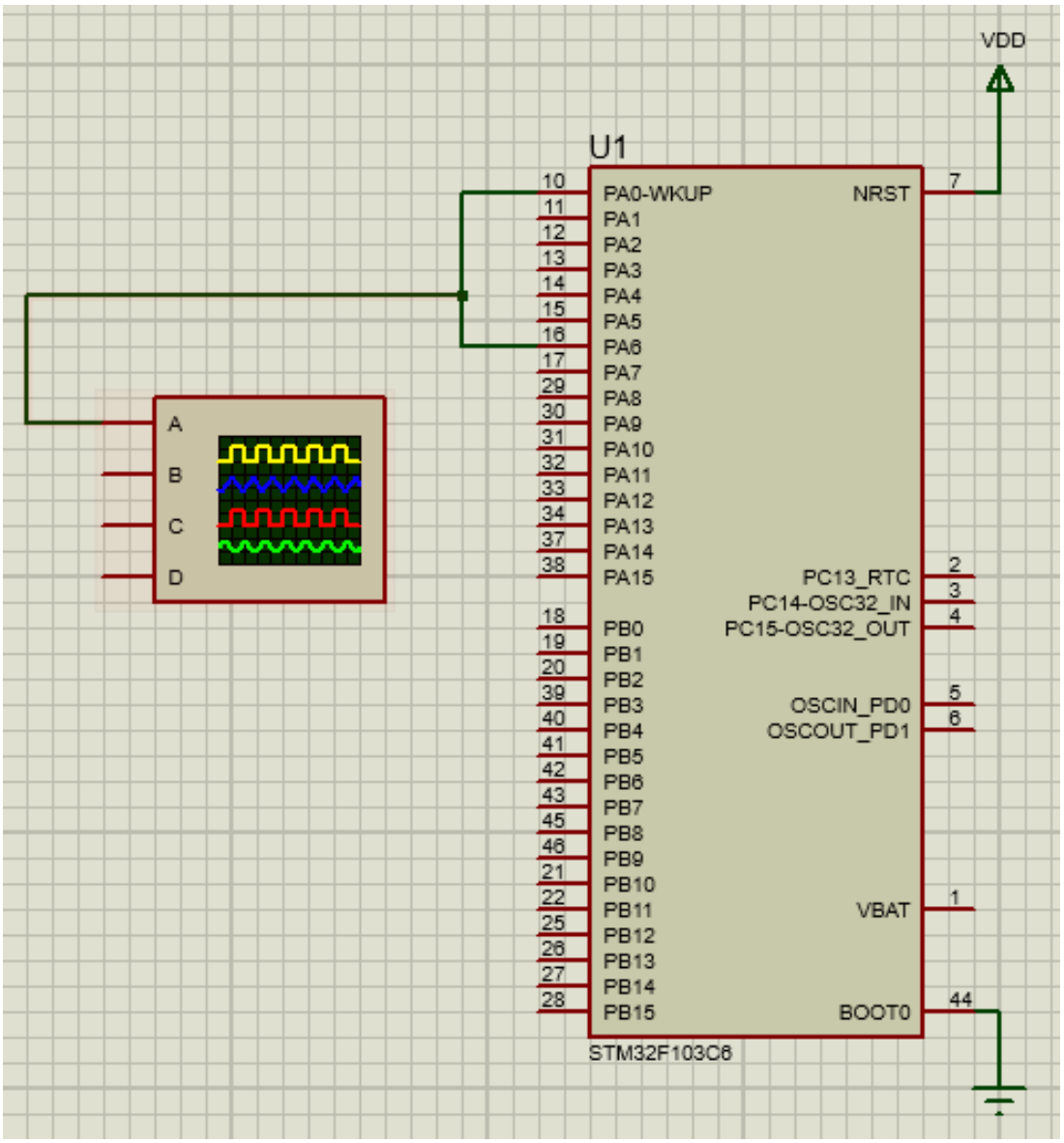


Рис. 6 – Схема для вимірювання періоду ШІМ-сигналу

Це завдання демонструє апаратне вимірювання періоду ШІМ-сигналу за допомогою таймера. Спочатку було налаштовано порти GPIO: пін PA0 (вихід TIM2_CH1) сконфігуровано в режимі альтернативної функції з виходом push-pull для генерації ШІМ-сигналу, а пін PA6 (вхід TIM3_CH1) – в режимі плаваючого входу для захоплення імпульсів.

Таймер TIM2 було налаштовано для генерації ШІМ з періодом 30 мкс та заповненням 30%. Через особливості симулятора Proteus реальна частота таймера відрізнялася від очікуваної, тому значення регістрів ARR та CCR1 було підібрано емпірично: ARR = 240 забезпечує 240 тіків таймера, що відповідає періоду 30 мкс, а CCR1 = 72 забезпечує 30% заповнення сигналу.

Таймер TIM3 було сконфігуровано в режимі захоплення імпульсів на каналі 1 для вимірювання періоду ШІМ. В обробнику переривань TIM3_IRQHandler фіксується значення лічильника у момент фронту сигналу, обчислюється різниця між поточним та попереднім захватом і зберігається у змінній period.

Результатом роботи програми є апаратна генерація стабільного ШІМ-сигналу на TIM2 та коректне вимірювання його періоду на TIM3, що на осцилографі відповідає періоду 30 мкс і скважності 30%.

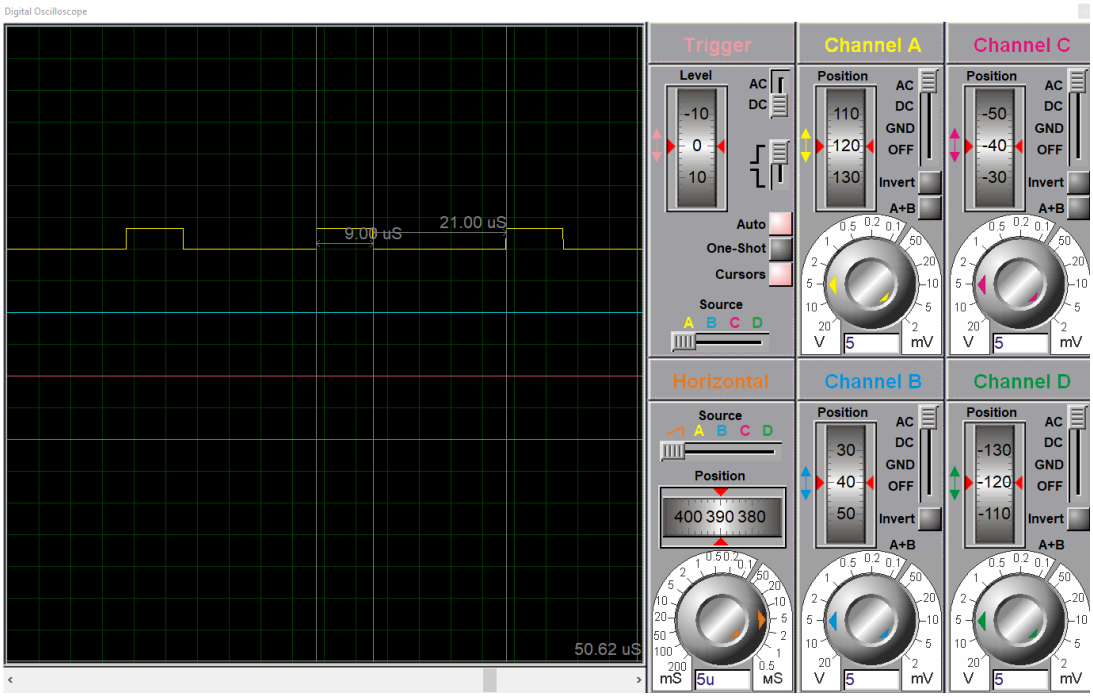


Рис. 7 – Результат таймінгів з осцилографа генерації та вимірювання ШІМ

```
#include "stm32f1xx.h"

// Ініціалізація периферії
void Peripherals_Init(void) {
   // Увімкнення тактування для GPIOA та альтернативних функцій
   RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_AFIOEN;
    // Увімкнення тактування для таймерів TIM2 (генерація PWM) та TIM3 (захоплення)
   RCC->APB1ENR |= RCC_APB1ENR_TIM2EN | RCC_APB1ENR_TIM3EN;

   // Налаштування PA0 як вихід PWM (TIM2_CH1)
```

```
  GPIOA->CRL &= ~(GPIO_CRL_CNF0 | GPIO_CRL_MODE0);      // Скидання бітів конфігурації
       GPIOA->CRL  |=  (GPIO_CRL_CNF0_1  |  GPIO_CRL_MODE0);          // Альтернативна функція push-pull, швидкість 50 МГц

   // Налаштування PA6 як вхід для захоплення PWM (TIM3_CH1)
  GPIOA->CRL &= ~(GPIO_CRL_CNF6 | GPIO_CRL_MODE6);      // Скидання бітів конфігурації
   GPIOA->CRL |= GPIO_CRL_CNF6_1;                       // Плаваючий вхід для захоплення

   // ==== Налаштування TIM2 для генерації PWM 30us / 30% ====
   TIM2->PSC = 0;          // Переддільник, один тик = 1/8МГц (Proteus)
   TIM2->ARR = 240;        // Період таймера 30 мкс
   TIM2->CCR1 = 72;        // Заповнення 30%
   TIM2->CCMR1 &= ~TIM_CCMR1_OC1M;
  TIM2->CCMR1 |= TIM_CCMR1_OC1M_2 | TIM_CCMR1_OC1M_1;  // Режим PWM1
   TIM2->CCER |= TIM_CCER_CC1E;                    // Увімкнення каналу
   TIM2->CR1 |= TIM_CR1_CEN;                       // Запуск таймера

   // ==== Налаштування TIM3 для захоплення PWM ====
   TIM3->PSC = 0;          // Переддільник
   TIM3->CCMR1 &= ~TIM_CCMR1_CC1S;
   TIM3->CCMR1 |= TIM_CCMR1_CC1S_0;   // CC1 mapped на TI1
    TIM3->CCER &= ~TIM_CCER_CC1P;       // Захоплення по зростаючому фронту
   TIM3->DIER |= TIM_DIER_CC1IE;      // Дозвіл переривання по захопленню
   TIM3->CR1 |= TIM_CR1_CEN;          // Запуск таймера
   NVIC_EnableIRQ(TIM3_IRQn);         // Дозвіл переривань у NVIC
}

// Змінні для обчислення періоду
volatile uint32_t last_capture = 0;  // Попереднє значення таймера при фронті сигналу
volatile uint32_t period = 0;        // Обчислений період сигналу

// Обробник переривання TIM3
void TIM3_IRQHandler(void) {
   if (TIM3->SR & TIM_SR_CC1IF) {          // Перевірка прапору захоплення
      TIM3->SR &= ~TIM_SR_CC1IF;           // Скидання прапору переривання
     uint32_t capture = TIM3->CCR1;     // Поточне значення таймера при фронті сигналу
       period = capture - last_capture;    // Обчислення періоду
       last_capture = capture;             // Запам’ятовування поточного значення
```

```
    }
}

int main(void) {
    Peripherals_Init();  // Ініціалізація периферії

    while(1) {
        // Основний цикл порожній, все працює через переривання
        // Змінна 'period' містить період сигналу в тактах таймера
    }
}
```

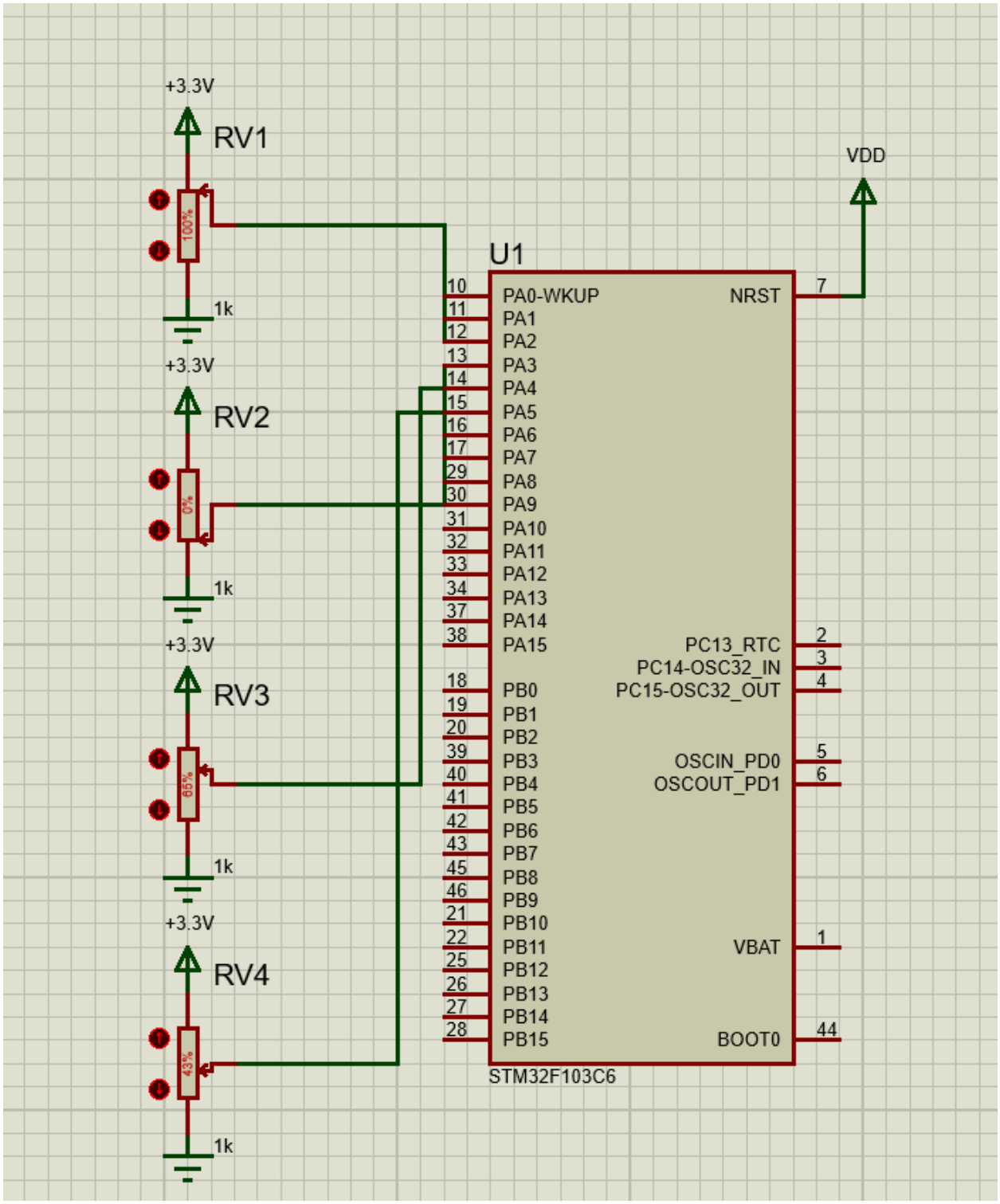
5. Написати програму розрахунку середньої напруги каналів АЦП регулярної групи через задані інтервали часу, для чого:
- налаштувати АЦП на режим неперервного вимірювання групи регулярних каналів, номери яких наведено у табл. 4. На відповідні канали АЦП подати вхідні напруги;
- налаштувати контролер прямого доступу до пам’яті, який зберігає виміряні АЦП дані в пам’яті;
- налаштувати обробник переривання системного таймера SysTick на часовий інтервал, наведений у табл. 4. В обробнику переривання розрахувати середнє значення напруг каналів.

Варіант	Канали АЦП	Часовий інтервал, мс
21	5,2,4,3	7

Відповідно до варіанту завдання було обрано чотири аналогові канали АЦП, на які подаються вхідні напруги від зовнішніх джерел. Вимірювання напруг виконується у неперервному режимі з використанням режиму сканування регулярної групи каналів.

Для підвищення ефективності обробки даних та зменшення навантаження на центральний процесор у роботі використано контролер прямого доступу до пам’яті DMA. Він забезпечує автоматичне збереження результатів вимірювання АЦП у буфер оперативної пам’яті без участі процесора.

Розрахунок середнього значення напруг виконується з фіксованим часовим інтервалом, який формується системним таймером SysTick. При кожному спрацюванні переривання SysTick відбувається обчислення середнього арифметичного значення напруг усіх каналів регулярної групи, що дозволяє отримувати актуальну усереднену інформацію про рівень сигналів.



Мікроконтролер STM32F103C6, до аналогових входів якого підключено чотири потенціометри. Потенціометри під’єднані між джерелом живлення 3.3 В та землею, а їх рухомі контакти підключені до входів АЦП, що відповідають каналам регулярної групи. Така схема дозволяє формувати плавно змінювану аналогову напругу в діапазоні від 0 до 3.3 В.

Кожен потенціометр подає незалежну вхідну напругу на відповідний канал АЦП, що забезпечує можливість одночасного вимірювання кількох сигналів. АЦП працює у режимі сканування, послідовно виконуючи

перетворення напруг на всіх підключених каналах. Отримані цифрові значення автоматично передаються до пам’яті за допомогою DMA.

Таким чином, схема забезпечує безперервне вимірювання аналогових сигналів та їх апаратну передачу до пам’яті мікроконтролера, що є необхідною умовою для подальшого програмного розрахунку середнього значення напруг.

Source Code

*main.c

```c
#include "stm32f1xx.h"
#define ADC_CHANNELS_COUNT 4

// Буфер для збереження результатів вимірювання АЦП
volatile uint16_t adc_buffer[ADC_CHANNELS_COUNT];
volatile uint16_t average_adc_value = 0;
void RCC_Init(void) {
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;   // Увімкнення тактування порту GPIOA
    RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;   // Увімкнення тактування ADC1
    RCC->AHBENR  |= RCC_AHBENR_DMA1EN;    // Увімкнення тактування DMA1
}
//Ініціалізація портів GPIO для аналогових входів
void GPIO_Init(void) {
    GPIOA->CRL &= ~(GPIO_CRL_CNF2 | GPIO_CRL_MODE2 |
                    GPIO_CRL_CNF3 | GPIO_CRL_MODE3 |
                    GPIO_CRL_CNF4 | GPIO_CRL_MODE4 |
                    GPIO_CRL_CNF5 | GPIO_CRL_MODE5);
    // Піни PA2–PA5 переведені в аналоговий режим
}

/*
 * Ініціалізація DMA для передачі даних з АЦП у пам’ять
 * Дані з регістру ADC1->DR автоматично записуються в adc_buffer
 */
void DMA_Init(void) {
    DMA1_Channel1->CPAR  = (uint32_t)&ADC1->DR;     // Адреса джерела даних (ADC)
    DMA1_Channel1->CMAR  = (uint32_t)adc_buffer;    // Адреса приймача (буфер у RAM)
    DMA1_Channel1->CNDTR = ADC_CHANNELS_COUNT;     // Кількість переданих значень

    DMA1_Channel1->CCR = 0;
    DMA1_Channel1->CCR |= DMA_CCR_MINC;             // Автоматичне збільшення адреси пам’яті
    DMA1_Channel1->CCR |= DMA_CCR_MSIZE_0;          // Розмір даних у пам’яті – 16 біт
    DMA1_Channel1->CCR |= DMA_CCR_PSIZE_0;          // Розмір даних у периферії – 16 біт
    DMA1_Channel1->CCR |= DMA_CCR_CIRC;             // Круговий режим роботи DMA

    DMA1_Channel1->CCR |= DMA_CCR_EN;               // Увімкнення каналу DMA
}

void ADC_Init(void) {
    ADC1->SQR1 = 0;
    ADC1->SQR3 = 0;
    ADC1->SQR3 |= (5 << 0) | (2 << 5) | (4 << 10) | (3 << 15);
    ADC1->SQR1 |= (3 << 20);        // Кількість перетворень = 4
    ADC1->CR1  |= ADC_CR1_SCAN;     // Увімкнення режиму сканування
    ADC1->CR2  |= ADC_CR2_CONT;     // Неперервний режим вимірювання
    ADC1->CR2  |= ADC_CR2_DMA;      // Передача даних через DMA
    ADC1->CR2 |= ADC_CR2_ADON;      // Увімкнення АЦП
    for (volatile int i = 0; i < 1000; i++); // Затримка для стабілізації

    ADC1->CR2 |= ADC_CR2_CAL;       // Запуск калібрування АЦП
    while (ADC1->CR2 & ADC_CR2_CAL);

    ADC1->CR2 |= ADC_CR2_ADON;      // Запуск перетворень
}
// Обробник переривання системного таймера SysTick, Виконує обчислення середнього значення з усіх каналів АЦП
void SysTick_Handler(void) {
    uint32_t sum = 0;

    for (int i = 0; i < ADC_CHANNELS_COUNT; i++) {
        sum += adc_buffer[i];
    }

    average_adc_value = sum / ADC_CHANNELS_COUNT;  // Обчислення середнього значення
}
int main(void) {
    RCC_Init();
    GPIO_Init();
    DMA_Init();
    ADC_Init();
     //Налаштування системного таймера SysTick на інтервал 7 мс
     //Частота HSI = 8 МГц
     //8000 тактів = 1 мс → 8000 × 7 = 56000 тактів
    SysTick_Config(56000);
    while (1) {
        // Основний цикл порожній, Уся логіка виконується через ADC, DMA та переривання SysTick
    }
}
```

Програмна реалізація починається з ініціалізації системи тактування, де вмикається тактування порту GPIOA, аналого цифрового перетворювача ADC1 та контролера DMA1. Після цього відповідні піни порту GPIOA налаштовуються в аналоговий режим, що дозволяє використовувати їх як входи АЦП без цифрової обробки сигналу.

Далі виконується налаштування DMA у круговому режимі роботи. DMA сконфігуровано таким чином, щоб автоматично передавати результати вимірювань з регістру даних АЦП у масив оперативної пам’яті. Завдяки цьому всі результати вимірювань зберігаються у буфері без участі центрального процесора.

Аналого цифровий перетворювач налаштовується на режим неперервного вимірювання з використанням режиму сканування регулярної групи каналів. Визначається послідовність перетворення каналів відповідно до варіанту завдання, а також вмикається передача результатів через DMA. Після виконання калібрування АЦП запускається безперервний процес перетворення аналогових сигналів у цифрові значення.

Системний таймер SysTick налаштовується на генерацію переривань з заданим часовим інтервалом. В обробнику переривання SysTick виконується підсумовування значень усіх каналів АЦП, що зберігаються у буфері DMA, та розрахунок їх середнього арифметичного значення. Отриманий результат зберігається у глобальній змінній та оновлюється через рівні проміжки часу, що забезпечує регулярний контроль середнього рівня напруг на входах АЦП.

Висновок.

У ході виконання лабораторної роботи було набуто практичних навичок програмування таймерів та контролера прямого доступу до пам’яті мікроконтролера STM32F103C6 у середовищі Proteus. Було реалізовано налаштування незалежного сторожового таймера для апаратного перезавантаження системи, організовано неблокуючу обробку часових інтервалів за допомогою системного таймера SysTick, виконано генерацію та вимірювання параметрів ШІМ сигналу з використанням таймерів загального призначення, а також налаштовано аналого цифровий перетворювач у режимі неперервного сканування регулярної групи каналів із застосуванням DMA для збереження результатів вимірювань у пам’яті та подальшого розрахунку середньої напруги через задані інтервали часу, що дозволило поглибити розуміння принципів роботи вбудованих периферійних модулів та переривань у мікроконтролерних системах.

Контрольні питання

3.1. Наведіть призначення сторожових таймерів.

Сторожові таймери призначені для підвищення надійності роботи вбудованих систем шляхом автоматичного перезавантаження мікроконтролера

у разі збоїв у виконанні програми. Вони контролюють коректність роботи програмного забезпечення та у випадку зависання програми або некоректної роботи процесора ініціюють апаратне скидання системи що дозволяє повернути мікроконтролер у працездатний стан без участі користувача.

3.2. Опишіть особливості роботи віконного таймера.

Віконний таймер є різновидом сторожового таймера який додатково контролює момент перезавантаження лічильника. Перезавантаження дозволяється лише у визначеному часовому вікні і якщо програма намагається скинути таймер занадто рано або занадто пізно відбувається скидання мікроконтролера. Такий принцип роботи дозволяє виявляти не лише зависання програми але й порушення часової логіки її виконання.

3.3. Опишіть особливості роботи сторожового таймера.

Сторожовий таймер працює незалежно від основного тактового генератора мікроконтролера та використовує власне джерело тактування. Для запобігання перезавантаженню програма повинна періодично оновлювати значення лічильника таймера. У разі відсутності такого оновлення протягом заданого інтервалу часу сторожовий таймер ініціює апаратне скидання мікроконтролера що забезпечує захист системи від програмних збоїв.

3.4. Наведіть принцип роботи таймера SysTick.

SysTick є системним таймером ядра Cortex M який використовується для формування періодичних переривань з фіксованим інтервалом часу. Він працює шляхом відліку заданої кількості тактів процесора після чого генерує переривання та автоматично перезавантажується. Таймер SysTick зазвичай застосовується для реалізації програмних затримок керування часовими інтервалами та організації системного лічильника часу.

3.5. Опишіть функціональні можливості базового таймера.

Базовий таймер призначений для відліку часу та формування періодичних подій у мікроконтролерних системах. Він може використовуватись для генерації переривань з заданою періодичністю реалізації часових затримок синхронізації виконання програмних задач та керування іншими периферійними модулями мікроконтролера. Базовий таймер не має каналів введення або виведення сигналів і працює лише як лічильник часу.

3.6. Перелічіть регістри базового таймера та назвіть їх призначення.

До основних регістрів базового таймера належать регістр керування який визначає режим роботи таймера регістр переддільника що задає коефіцієнт поділу тактової частоти регістр автоматичного перезавантаження який визначає максимальне значення лічильника регістр лічильника який

зберігає поточне значення відліку та регістр статусу який містить прапори подій зокрема прапор переповнення таймера.

3.7. Опишіть особливості налаштування режиму захоплення сигналу.

Режим захоплення сигналу дозволяє фіксувати значення лічильника таймера у момент зміни стану зовнішнього сигналу. Для цього відповідний канал таймера налаштовується як вхід захоплення обирається фронт сигналу та за необхідності вмикається переривання. При появі фронту значення лічильника зберігається у регістрі захоплення що дозволяє вимірювати період частоту або тривалість імпульсів сигналу.

3.8. Опишіть особливості налаштування режиму ШІМ.

Режим широтно імпульсної модуляції реалізується шляхом порівняння значення лічильника таймера з заданим значенням у регістрі порівняння. Для налаштування ШІМ необхідно визначити період сигналу за допомогою регістра автоматичного перезавантаження та коефіцієнт заповнення за допомогою регістра порівняння. Додатково налаштовується режим каналу та вихід таймера що дозволяє формувати сигнал із заданими параметрами.

3.9. Наведіть призначення контролера прямого доступу до пам’яті.

Контролер прямого доступу до пам’яті призначений для автоматичної передачі даних між периферійними модулями та оперативною пам’яттю без участі центрального процесора. Його використання дозволяє зменшити навантаження на процесор підвищити швидкодію системи та забезпечити ефективну обробку великих обсягів даних зокрема при роботі з аналого цифровими перетворювачами таймерами та інтерфейсами обміну даними.

3.10. Опишіть принцип налаштування контролера прямого доступу до пам’яті.

Налаштування контролера прямого доступу до пам’яті передбачає вибір каналу передачі визначення адреси джерела та адреси приймача задання кількості переданих даних та конфігурацію параметрів передачі. Додатково визначається напрямок передачі розмір даних режим інкремента адрес та тип роботи зокрема одноразовий або круговий режим. Після завершення конфігурації канал DMA активується та починає автоматичну передачу даних відповідно до заданих параметрів.